



实验四：双能谱图像的校正实践

目录

-
- 1 期末大作业——双能谱图像的校正实践
 - 2 深度学习图像去噪方法
 - 3 图像去噪——卷积神经网络
 - 4 基于卷积神经网络的手写数字识别

期末大作业——双能谱图像的校正实践

➤ 1、问题背景:

医疗成像设备采集**双能谱数据**，通过某种成熟的数据处理方法得到基质图像，这些基质图像成对产生，对应同一组织中不同基质的结构。这一过程可以概括为： $\begin{pmatrix} d_1 \\ d_2 \end{pmatrix} \rightarrow \begin{pmatrix} f \\ g \end{pmatrix}$ 。其中 d_1, d_2 表示双能谱数据， (f, g) 则是成对产生的基质图像，“ $\rightarrow$ ”代表某种成熟的数据处理方法。由于设备采集到的数据 d_1, d_2 必定含有噪声，而数据处理方法“ $\rightarrow$ ”往往具有很多局限性，得到的基质图像 (f, g) 与真实图像 (f^*, g^*) 可能有较大偏差。

➤ 2、项目任务:

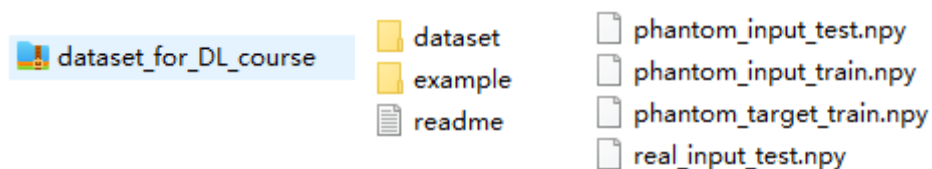
运用深度学习方法对图像结果 (f, g) 进行提升，使之更接近于真实图像 (f^*, g^*) ，在测试集上测试其泛化能力。

本项目可以近似地看作是一个**去噪任务**，即真实图像 (f^*, g^*) 被某种未知噪声干扰，需要从有噪声的图像结果 (f, g) 中恢复真实图像。我们需要运用深度学习的方法学习这种干扰特征，并对图像结果进行提升。

期末大作业——双能谱图像的校正实践

➤ 3、数据集dataset_for_DL_course:

解压文件夹后可以参见readme.txt，用于训练和预测的数据集都在dataset文件夹中。



(1) phantom_input_train.npy 给出2000组噪声模拟图(f, g), 用于训练神经网络, 形状为(2000, 2, 256, 256), 每张图片的像素为256*256;

(2) phantom_target_train.npy 给出2000组无噪声模拟图, 为上述噪声图像(f, g)一一对应的真实图像(f^*, g^*), 用于训练神经网络, 形状为(2000, 2, 256, 256), 每张图片的像素为256*256;

(3) phantom_input_test.npy 和 real_input_test.npy 给出用于测试的噪声图, 分别是25组噪声模拟图（与训练集中的相似）以及25组噪声图（来源于经过处理的真实医疗数据集），用于测试神经网络, 形状都为(25, 2, 256, 256)。

(4) 数据集读取示例:

```
import numpy
phantom_input_train = numpy.load('path-to-your-dataset/dataset/phantom_input_train.npy')
```

期末大作业——双能谱图像的校正实践

➤ 4、项目要求：

- ◆ 本项目需要运用 `phantom_input_train.npy` 和 `phantom_target_train.npy` 训练神经网络，然后将训练好的神经网络用于测试集 `phantom_input_test.npy` 和 `real_input_test.npy`，得到提升后的结果。
- ◆ 其中 `phantom_input_test.npy` 与训练集相似，应该能得到很好的结果；`real_input_test.npy`上的预测有很大挑战，主要用于观察网络的泛化能力。
- ◆ 同学们可以自行选择、设计神经网络结构，自主合理使用数据集，寻求一切可用的计算资源来完成任务。
- ◆ 编写项目报告，格式如下：

——题目：自取

——摘要

——关键词

—1. 图像去噪的背景及意义

—2. 深度学习图像去噪方法

—3. 本文使用的方法

—4. 主要运行步骤

—5. 运行结果或结论

—6. 总结(困难或调试)

—附录（代码）

期末大作业——双能谱图像的校正实践

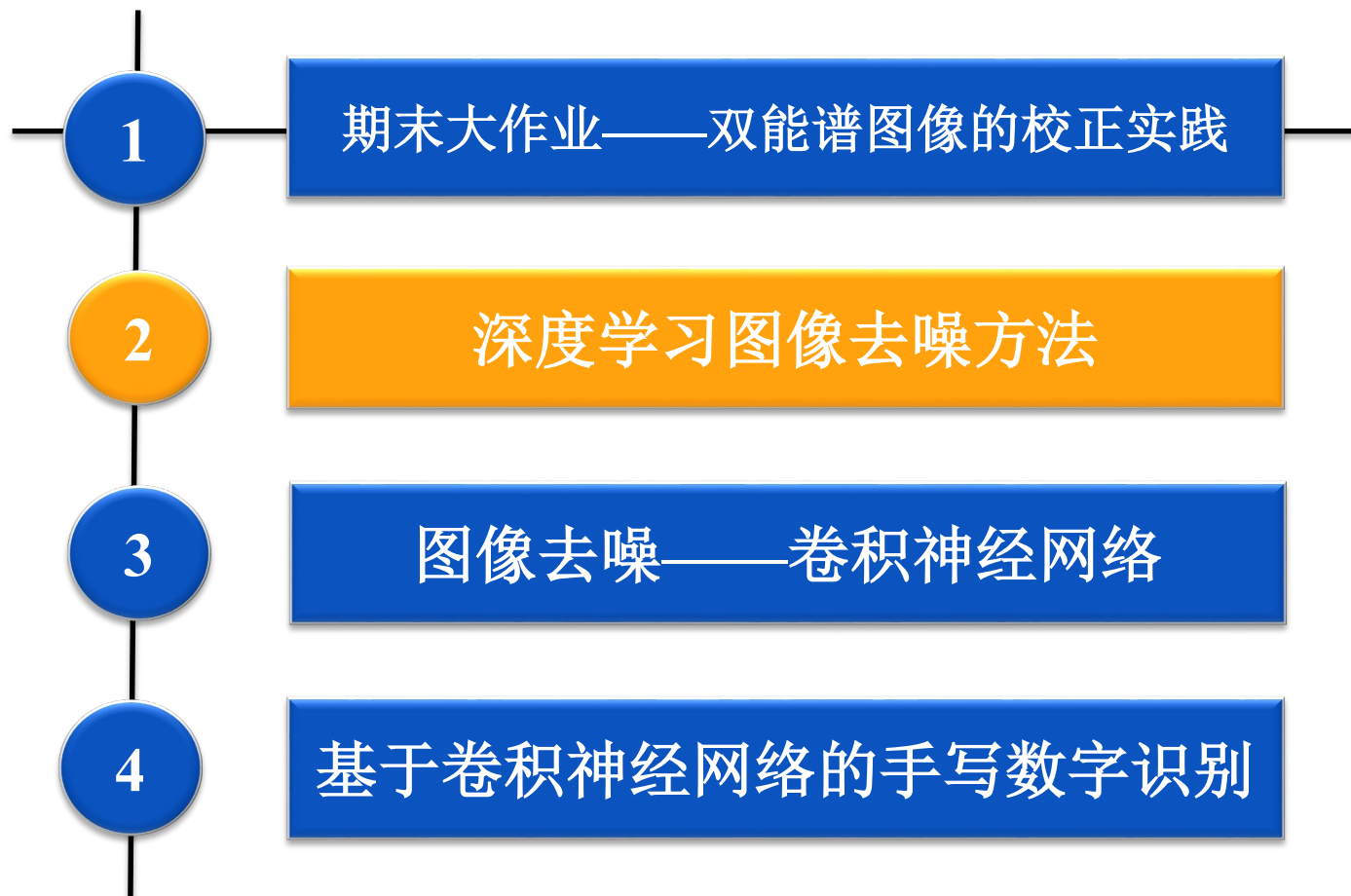
➤ 5、提交方式:

- 将项目报告.pdf文件和预测结果图片（50张）放置在一个文件夹内，打包成zip压缩包，提交至平台 <https://grader.hitsz.edu.cn/courses>

注意:

- 1、用户名用统一身份登录;
- 2、请提交到相应的条目「2026春-深度学习」课程 - 实验四;
- 3、提交截止时间: 7月5号 晚24点;
- 4、文件夹&压缩包命名要求: 学号_姓名_深度学习实验四
- 5、提交内容: 项目报告(.pdf文件) + 代码(.py/ipynb文件, 重要代码处加注释)
+预测结果图片（50张），打包成zip压缩包提交。

目录

- 
- 1 期末大作业——双能谱图像的校正实践
 - 2 深度学习图像去噪方法
 - 3 图像去噪——卷积神经网络
 - 4 基于卷积神经网络的手写数字识别

深度学习图像去噪方法

➤ 1、卷积神经网络（CNN）

- **基本原理：**CNN能够通过卷积层提取图像特征，并通过多层网络结构逐步去除图像中的噪声。卷积层可以学习图像的空域特征，而池化层可以降低图像的分辨率，从而减少参数数量和计算复杂度。
- **代表性模型：****DnCNN**（Denoising Convolutional Neural Network）是一个经典的CNN模型，它在图像去噪任务中展示了优异性能。DnCNN模型由多个卷积层、激活函数和批归一化层组成，可以高效地去除不同类型的噪声。

➤ 2、自编码器（Autoencoder）

- **基本原理：**自编码器是一种无监督学习方法，通常用于降噪。它包括一个编码器和一个解码器，编码器将图像编码为低维潜在表示，解码器则将其重构为原始图像。在去噪过程中，自编码器学习如何在重构图像时去除噪声，从而得到去噪后的图像。
- **代表性模型：****去噪自编码器**（Denoising Autoencoder）专门针对图像去噪进行了优化。

➤ 3、生成对抗网络（GAN）

- **基本原理：**网络通过框架中的生成器（Generative Model）和判别器（Discriminative Model）的互相博弈学习产生输出。生成器生成去噪后的图像，判别器则区分去噪图像和真实图像
- **代表性模型：****Deep Convolutional GAN (DCGAN)** 模型

目录

- 1 期末大作业——双能谱图像的校正实践
- 2 深度学习图像去噪方法
- 3 图像去噪——卷积神经网络
- 4 基于卷积神经网络的手写数字识别

图像去噪——卷积神经网络

一、常用卷积神经网络算法及其结构

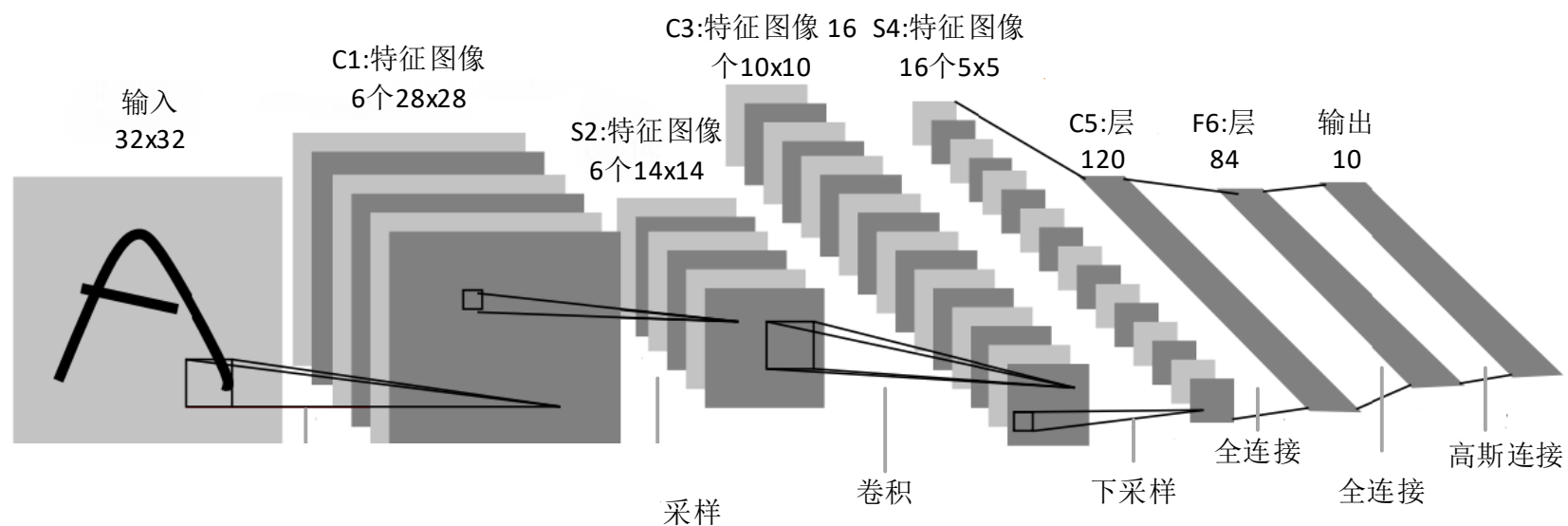
- 1. LeNet5
- 2. AlexNet
- 3. VGGNet
- 4. GoogLeNet
- 5. ResNet

图像去噪——卷积神经网络

一、常用卷积神经网络算法及其结构

➤ 1. LeNet5

LeNet5是杨立昆（Yann LeCun）在1998年设计的用于手写数字识别的卷积神经网络，当年大多数银行就是用LeNet5来识别支票上面的手写数字的，是早期卷积神经网络中最有代表性的实验系统之一。LeNet5共有7层（不包括输入层），每层都包含不同数量的训练参数，其网络结构如图所示。



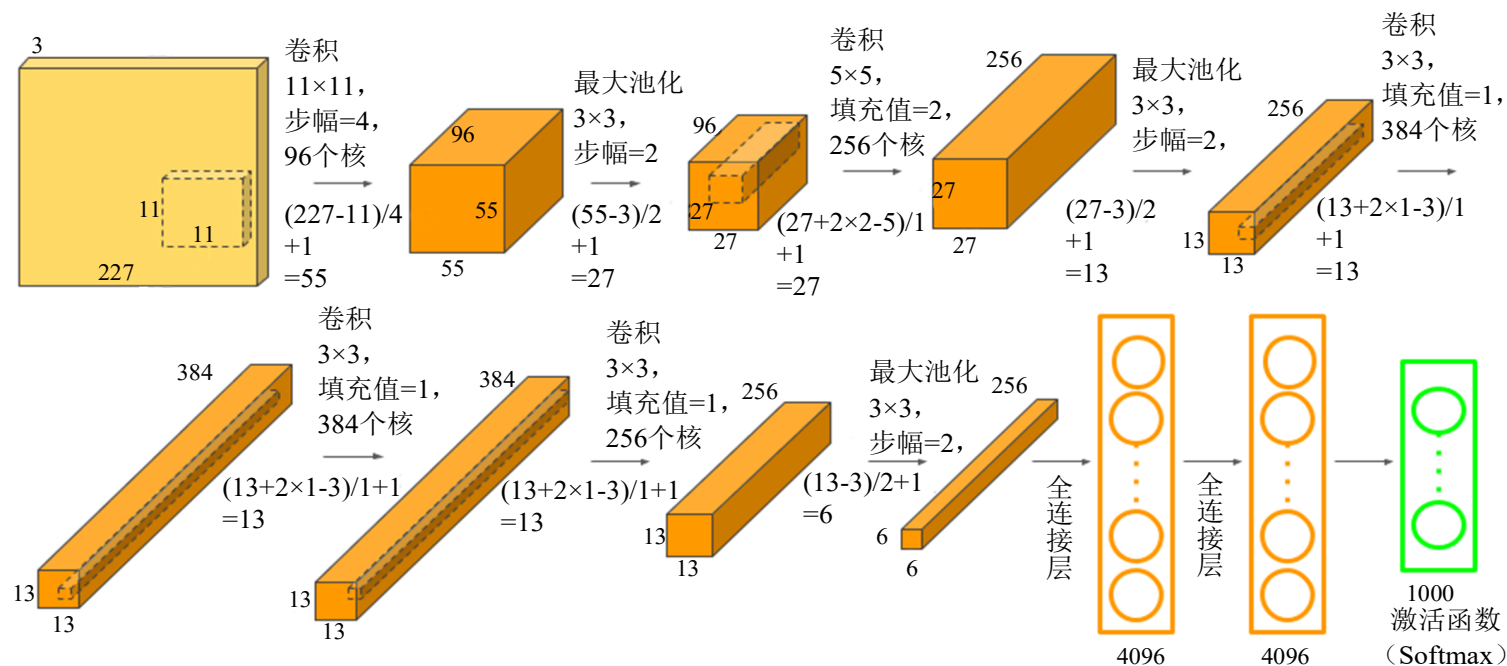
图像去噪——卷积神经网络

一、常用卷积神经网络算法及其结构

➤ 2. AlexNet

AlexNet于2012年由阿莱克斯·克里泽夫斯基（Alex Krizhevsky），伊尔亚·苏茨克维（Ilya Sutskever）和杰弗里·辛顿（Geoffrey Hinton）等人提出。AlexNet网络一共分为11层，含5个卷积层以及3个全连接层，除此之外还有3个池化层。在每一个卷积层中包含了激活函数RELU以及局部响应归一化（LRN）处理，然后经过降采样（池化处理）。其网络结构如图所示。

图中输入的图片尺寸为 256×256 ，然后对输入的图片进行随机裁剪得到 227×227 大小的图片，将图片输入进网络中，最后得到1000个分布区间为 $(0,1)$ 的数的输出，代表输入样本的所属类别的概率。





图像去噪——卷积神经网络

一、常用卷积神经网络算法及其结构

➤ 3. VGGNet

VGGNet（Visual Geometry Group）于2014年由牛津大学的凯伦·西蒙扬（Karen Simonyan）和安德鲁·齐瑟曼（Andrew Zisserman）提出，主要特点是“简洁与深度”。简洁是指它的结构一律采用步长（stride）为1的 3×3 的卷积核（filter），以及步长为2的 2×2 的最大池化（MaxPooling），深度则是因为VGGNet能够达到19层。

VGGNet一共有6种不同的网络结构，每种结构都含有5组卷积，每组卷积都使用 3×3 的卷积核，每组卷积后进行一个 2×2 的最大池化，然后是3个全连接层。VGGNet的示意图如图所示，其中网络结构D就是著名的VGG16，网络结构E就是著名的VGG19。

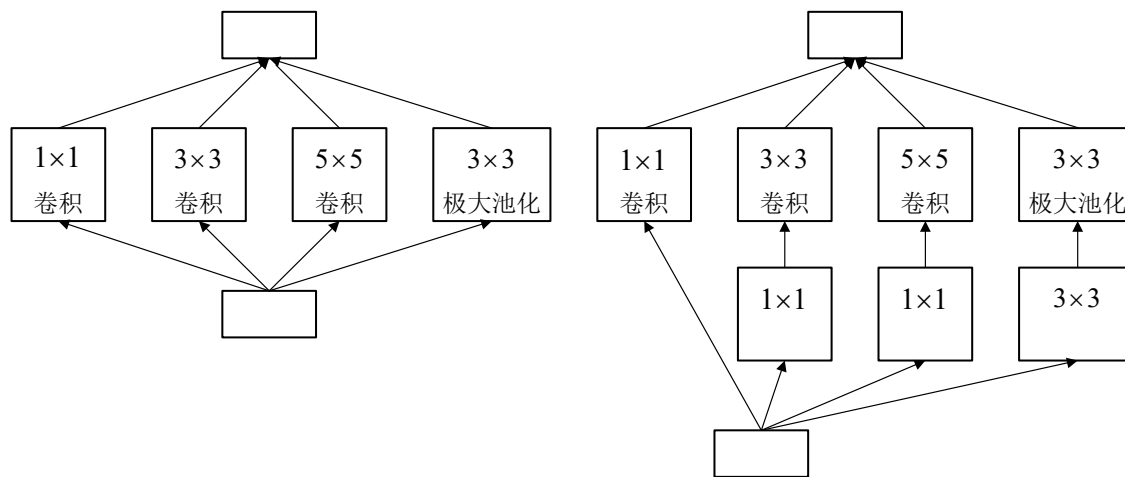
ConvNet Configuration					
A	A-LRN	B	C	D	E
11 权重	11 权重	13 权重	16 权重	16 权重	19 权重
输入(224 x 224 RGB 图片)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
最大池化层					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
最大池化层					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256
最大池化层					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512
最大池化层					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512
最大池化层(maxpooling)					
全连接层-4096(FC-4096)					
全连接层-4096(FC-4096)					
全连接层-1000 (FC-1000)					
激活函数 (Softmax)					

图像去噪——卷积神经网络

一、常用卷积神经网络算法及其结构

➤ 4. GoogLeNet

GoogLeNet是2014年由克里斯提·鲁布托（Christian Louboutin）提出的一种全新的深度学习网络结构，在这之前的AlexNet、VGG等结构都是通过增大网络的深度（层数）来获得更好的训练效果，但层数的增加会带来很多副作用，如过拟合、梯度消失、梯度爆炸等。网络宽度（inception）则从另一种角度来提升训练结果，能更高效地利用计算资源，在相同的计算量下能提取到更多的特征。GoogLeNet的inception结构如图所示。

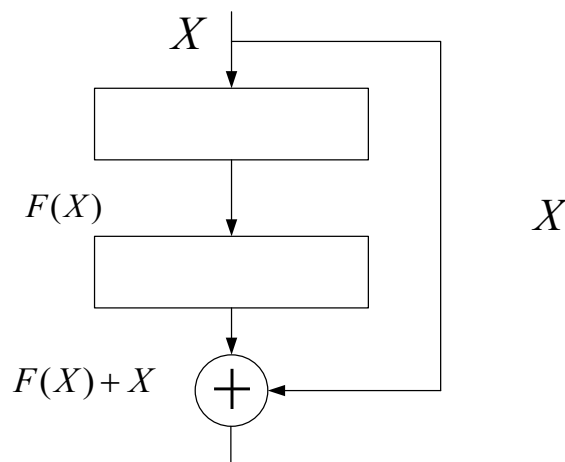


图像去噪——卷积神经网络

一、常用卷积神经网络算法及其结构

➤ 5. ResNet

随着层数的增加，CNN的效果会遇到瓶颈，甚至会不增反降。ResNet于2015年由微软亚洲研究院的学者们提出，主要是为了解决这个问题，帮助训练更深的网络。ResNet引入了一个残差块（residual block）的结构，如图所示。



图像去噪——卷积神经网络

二、卷积神经网络中的常用网络层

- 1. 卷积层
- 2. 池化层
- 3. 全连接层
- 4. 归一化层
- 5. 丢弃层

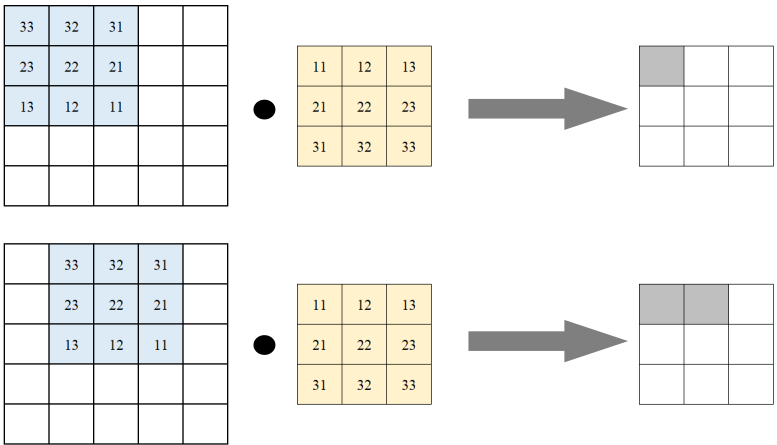
图像去噪——卷积神经网络

二、卷积神经网络中的常用网络层

➤ 1. 卷积层

卷积的实现过程

在局部感知和权值共享的基础上，网络中的每一层的计算操作是输入层和权重的卷积，卷积神经网络的名字因此而来。设定一个大小为 5×5 的图像和一个 3×3 的卷积核。此处的卷积核共有9个参数。这种情况下，卷积核实际上有9个神经元，它们的输出又组成一个 3×3 的矩阵，称为特征图。卷积的过程如图所示。



第一个神经元连接到图像的第三个 3×3 的局部，第二个神经元则滑动连接到第二个 3×3 的局部，总共需要滑动8次。



图像去噪——卷积神经网络

二、卷积神经网络中的常用网络层

➤ 1. 卷积层

PyTorch框架中的常用卷积类型

① Conv2d

Conv2d类的语法格式如下。

```
torch.nn.Conv2d(in_channels, out_channels,
kernel_size, stride=1, padding=0, dilation=1,
groups=1, bias=True, padding_mode='zeros')
```

Conv2d类创建一个卷积核，该卷积核对层输入进行卷积，以生成输出张量。如果参数bias的值为True，则会创建一个偏置向量并将其添加到输出中。

参数名称	说明
in_channels	接收int，表示输入通道数，无默认值
out_channels	接收int，表示输出通道数，无默认值
kernel_size	接收int或tuple，表示卷积核尺寸，无默认值
stride	接收int或tuple，表示卷积操作的步幅，默认为1
padding	接收int或tuple，表示用于在输入数据中填充的层数，默认为0
dilation	接收int或tuple，表示卷积核内部各点的间距，默认为1
groups	接收int，表示控制输入和输出之间的连接；group=1，输出是所有输入的卷积；group=2，此时相当于有并排的两个卷积层，每个卷积层计算输入通道的一半，并且产生的输出是输出通道的一半，随后将这两个输出连接起来，默认为1
bias	接收bool，为true表示对输出添加可学习的偏置量，默认为True
padding_mode	接收str，表示填充模式，默认为'zeros'



图像去噪——卷积神经网络

二、卷积神经网络中的常用网络层

➤ 1. 卷积层

PyTorch框架中的常用卷积类型

② ConvTranspose2d

ConvTranspose2d类的语法格式如下。

```
torch.nn.ConvTranspose2d(in_channels, out_channels, kernel_size, stride=1, padding=0, output_padding=0, groups=1, bias=True, dilation=1, padding_mode='zeros')
```

ConvTranspose2d类的常用参数及其说明与Conv2d类一致。增加的参数output_padding接收int或tuple，表示对输出的hw填充的层数。



图像去噪——卷积神经网络

二、卷积神经网络中的常用网络层

➤ 1. 卷积层

PyTorch框架中的常用卷积类型

③ Conv3d

Conv3d类的语法格式如下。

`torch.nn.Conv3d(in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1, groups=1, bias=True)`

Conv3d类的常用参数及其说明与Conv2d一致。

二、卷积神经网络中的常用网络层

➤ 2. 池化层

在卷积层中，可以通过调节步长参数来达到减少输出尺寸的目的，池化层同样基于局部相关性的思想，在局部相关的一组元素中进行采样或信息聚合，从而得到新的元素值。如最大池化层（**Max Pooling**）返回局部相关元素集中最大的元素值，平均池化层（**Average Pooling**）返回局部相关元素集中元素的平均值。

池化（**pool**）即下采样（**downsamples**），目的是减少特征图的尺寸。池化操作对于每个卷积后的特征图是独立进行的，池化窗口规模一般为 2×2 ，相对于卷积层进行卷积运算，池化层进行的运算一般有以下3种。

- （1）最大池化（**Max Pooling**）。取4个元素的最大值，这是最常用的池化方法。
- （2）平均值池化（**Mean Pooling**）。取4个元素的均值。
- （3）高斯池化。借鉴高斯模糊的方法。

如果池化层的输入单元大小不是2的整数倍，一般采取边缘补零（**zero-padding**）的方式补成2的倍数，然后再池化。



图像去噪——卷积神经网络

二、卷积神经网络中的常用网络层

➤ 2. 池化层

(1) MaxPool2d

二维最大池化MaxPool2d的作用是对二维信号（图像）进行最大值池化，其计算过程如图所示，其中池化窗口大小为(2,2)，步长大小为(2,2)，由一个大小为 4×4 的区域下采样得到一个大小为 2×2 的区域。



图像去噪——卷积神经网络

二、卷积神经网络中的常用网络层

➤ 2. 池化层

(1) MaxPool2d 语法格式如下。

```
torch.nn.MaxPool2d(kernel_size, stride=None, padding=0, dilation=1, return_indices=False, ceil_mode=False)
```

参数名称	说明
kernel_size	接收int或tuple，表示池化核尺寸，无默认值
stride	接收int或tuple，表示窗口的步幅，默认就等于窗口尺寸，默认为None
padding	接收int或tuple，表示数据hw方向上的零填充层数，默认为0
dilation	接收int或tuple，表示一个控制窗口中元素步幅的参数，默认为1
return_indices	接收bool，表示如果等于True，会返回输出最大值的序号，对于上采样操作会有帮助，默认为False
ceil_mode	接收bool，表示如果等于True，计算输出数据的hw的时候，会使用向上取整，代替默认的向下取整的操作，默认为F
kernel_size	接收int或tuple，表示池化核尺寸，无默认值

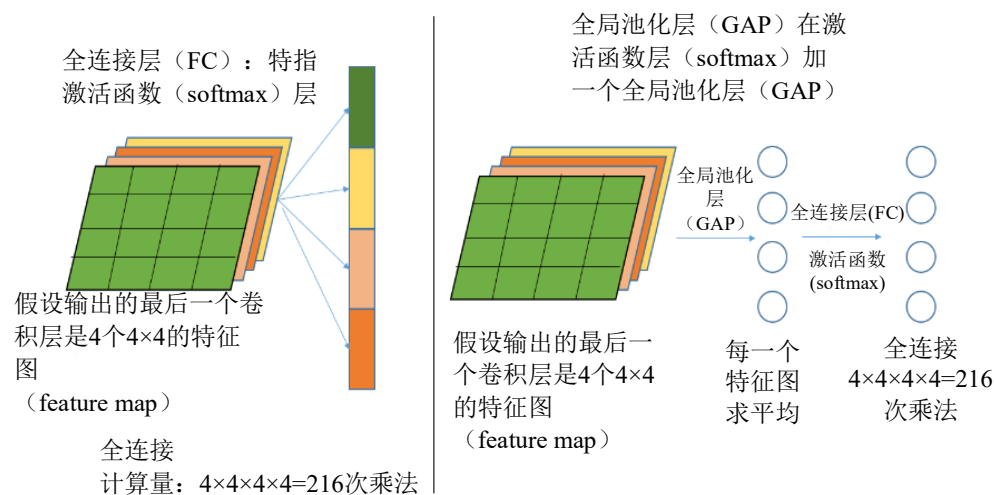
图像去噪——卷积神经网络

二、卷积神经网络中的常用网络层

➤ 2. 池化层

(2) AvgPool2d

AvgPool2d表示对二维信号(图像)进行平均值池化。对于输出的每一个通道的特征图的所有像素计算一个平均值，经过全局平均池化（Global Average Pooling，GAP）之后得到一个特征向量，该向量的维度表示类别数，然后直接输入到softmax层。全局平均池化的图解如图所示。





图像去噪——卷积神经网络

二、卷积神经网络中的常用网络层

➤ 2. 池化层

(2) AvgPool2d 语法格式如下。

```
torch.nn.AvgPool2d(kernel_size, stride=None, padding=0, ceil_mode=False, count_include_pad=True, divisor_override=None)
```

参数名称	说明
kernel_size	接收int或tuple，表示池化核尺寸，无默认值
stride	接收int或tuple，表示窗口的步幅，默认就等于窗口尺寸，默认为None
padding	接收int或tuple，表示数据hw方向上的零填充层数，默认为0
ceil_mode	接收bool，表示如果等于True，计算输出数据的hw的时候，会使用向上取整，代替默认的向下取整的操作，默认为False
count_include_pad	接收bool，表示如果等于True，计算平均池化时，将包括padding填充的0，默认为True
divisor_override	接收int，表示求平均值时，可以不使用像素值的个数作为分母，而是使用除法因子，默认为None

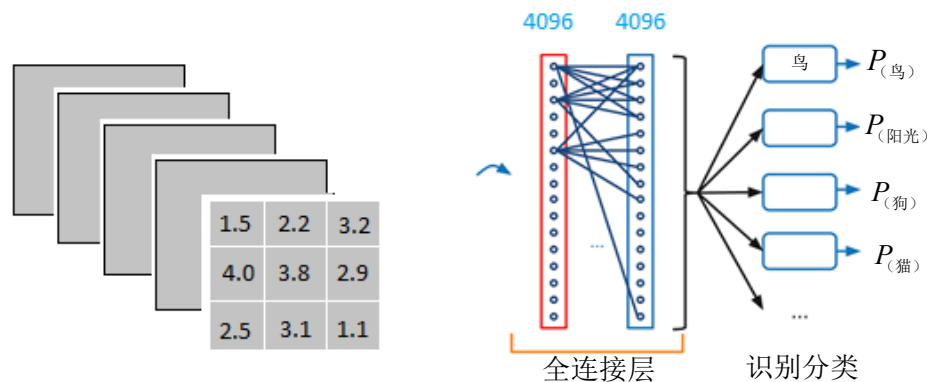
图像去噪——卷积神经网络

二、卷积神经网络中的常用网络层

➤ 3. 全连接层

全连接层是指每个神经元与前一层所有的神经元全部连接，在整个卷积神经网络中起到“分类器”的作用，即通过卷积、激活函数、池化等深度网络提取出图像的特征后，再经过全连接层对结果进行识别分类。

例如，得到的特征图的大小是 $3 \times 3 \times 5$ ，然后将特征图中的所有神经元变成全连接层，实际上就是将一个3D的立方体重新排列，变成一个全连接层，里面包含有 1×4096 个神经元，经过一个或多个隐藏层后输出结果，如图所示。在这个过程中为了防止过拟合会引入丢弃层Dropout。最近的研究也表明，在进入全连接层之前，使用全局平均池化能够有效地降低过拟合。

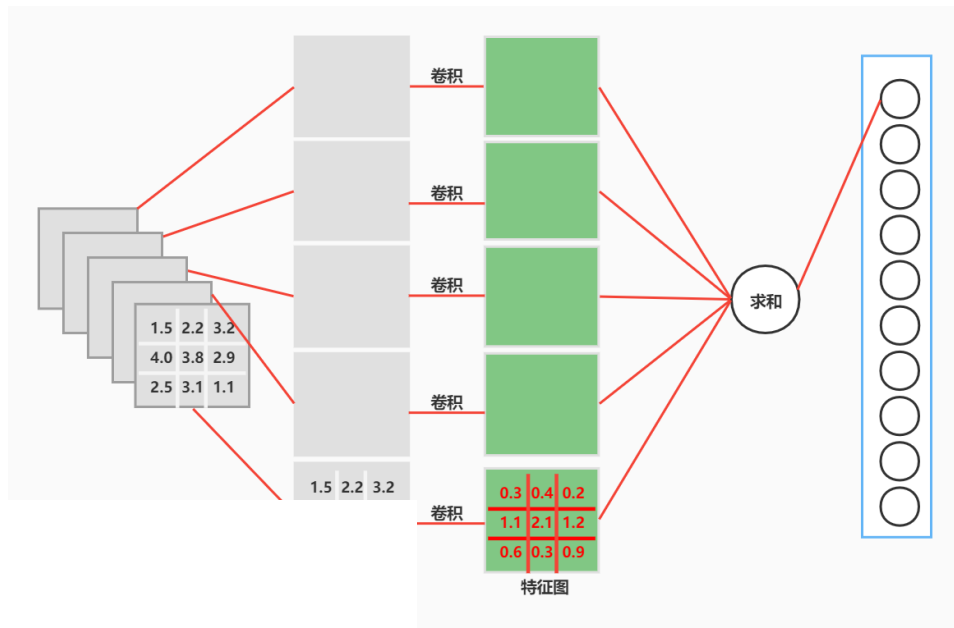


图像去噪——卷积神经网络

二、卷积神经网络中的常用网络层

3. 全连接层

要将 $3 \times 3 \times 5$ 的输出转换成 1×4096 的形式，可以理解为在中间做了一个卷积，用一个 $3 \times 3 \times 5$ 的卷积核去卷积激活函数的输出，得到的结果就是全连接层的一个神经元的输出，这个输出就是一个值。因为有4096个神经元，所以就是用一个 $3 \times 3 \times 5 \times 4096$ 的卷积层去卷积激活函数的输出。全连接层的计算过程，如图所示。



二、卷积神经网络中的常用网络层

➤ 3.全连接层

PyTorch的Linear类用于设置网络中的全连接层，语法格式如下。

```
torch.nn.Linear(in_features, out_features, bias=True)
```

Linear类的常用参数及其说明如表所示。

参数名称	说明
in_features	接收int，表示上层神经元个数，无默认值
out_features	接收int，表示本层神经元个数，无默认值
bias	接收bool类，表示偏置，形状为[out_features]。如果设置bias = False，那么该层将不会学习一个加性偏差。默认为True

二、卷积神经网络中的常用网络层

➤ 4. 归一化层

归一化层利用小批量上的均值和标准差，不断调整网络的中间输出，从而使整个网络在各层的中间输出的数值更稳定，提高训练网络的有效性。

归一化层的主要作用是防止梯度爆炸和梯度消失。目前归一化层主要有5种：批次归一化层（Batch Normalization, BN）、横向归一化层（Layer Normalization, LN）、实例归一化层（Instance Normalization, IN）、组归一化层（Group Normalization, GN）和可切换归一化层（Switchable Normalization, SN）。

深度网络中的数据维度一般是 $[N, C, H, W]$ 或者 $[N, H, W, C]$ 格式， N 是batch size， H/W 是特征（feature）的高/宽， C 是feature的通道（channel），压缩 H/W 至一个维度，其中4种常用的归一化层的三维表示如图所示。

图像去噪——卷积神经网络

二、卷积神经网络中的常用网络层

➤ 4.归一化层

BatchNorm2d类的语法格式如下。

```
torch.nn.BatchNorm2d(num_features, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
```

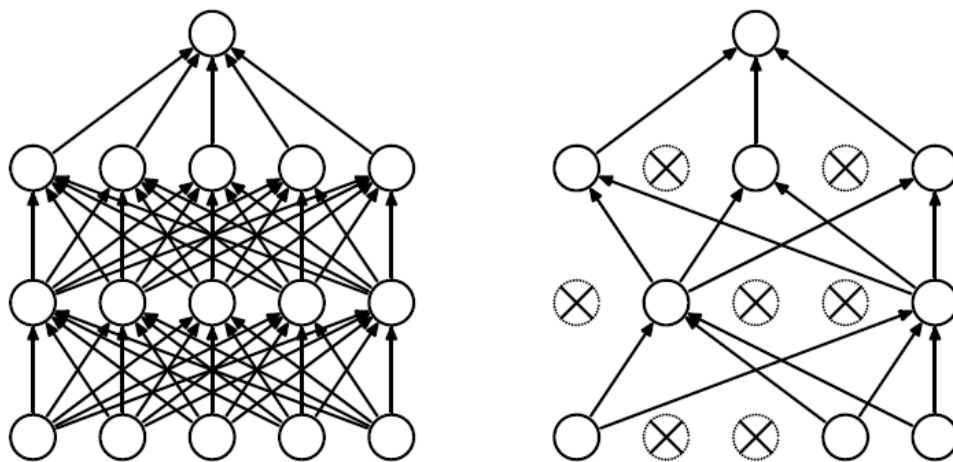
参数名称	说明
in_features	接收int，表示上层神经元个数，无默认值
out_features	接收int，表示本层神经元个数，无默认值
bias	接收bool类，表示偏置，形状为[out_features]。如果设置bias = False，那么该层将不会学习一个加性偏差。默认为True

图像去噪——卷积神经网络

二、卷积神经网络中的常用网络层

➤ 5. 丢弃层

在前向传播的时候，让某个神经元的激活值以一定的概率（**rate**）停止工作，这样可以使模型泛化能力更强，因为神经元不会太依赖某些局部的特征，丢弃层（**Dropout**）的工作原理如图所示。首先随机（临时）删掉网络中一些隐藏层的神经元，得到修改后的网络；然后对一小批输入数据前向传播，再将得到的损失通过修改后的网络反向传播，按照随机梯度下降法更新对应的参数（只更新没有被删掉的神经元的权重）；最后恢复被删掉的神经元，重复此过程。



图像去噪——卷积神经网络

二、卷积神经网络中的常用网络层

➤ 5. 丢弃层

Dropout类的语法格式如下。

`torch.nn.Dropout(p=0.5, inplace=False)`

Dropout类的常用参数及其说明如表所示。

参数名称	说明
p	接收float，表示张量元素被置0的概率，默认为0.5
inplace	接收int，表示是否原地执行，默认为False

目录

- 1 期末大作业——双能谱图像的校正实践
- 2 深度学习图像去噪方法
- 3 图像去噪——卷积神经网络
- 4 基于卷积神经网络的手写数字识别

基于卷积神经网络的手写数字识别

下面将使用卷积神经网络实现手写数字识别图像分类。使用的数据集为MNIST数据集，是一个大型的手写体数字数据库，通常用于训练各种图像处理系统，也被广泛用于机器学习领域的训练和测试。MNIST数据集共有训练数据60000项、测试数据10000项。每张图像的大小为 28×28 （像素），每张图像都为灰度图像，位深度为8（灰度图像是0-255）。下面从导入相关库、数据加载与预处理、构建网络、训练网络和性能评估等方面进行介绍。

基于卷积神经网络的手写数字识别

➤ 1. 导入相关库

导入一些必要的包，除了常用的torch和Variable之外，还导入了DataLoader用于加载数据，使用torchvision对图像进行预处理。

➤ 2. 数据加载与预处理

首先，定义超参数，如代码所示。n_epochs定义了整个训练数据集的循环次数，learning_rate和momentum是后续使用的优化器的超参数，random_seed是为可重复的实验设置并使用随机数产生设置随机种子。

其次，加载MNIST数据集。Normalize函数对数据进行标准化，使用的值为0.1307和0.3081，是MNIST数据集的全局平均值和标准偏差。

```
# 代码1
# 导入相关库
import torch
import torchvision
from torch.autograd import Variable
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
```

```
# 代码2
# 超参数设置
n_epochs = 3
batch_size_train = 64
batch_size_test = 1000
learning_rate = 0.01
momentum = 0.5
log_interval = 10
random_seed = 1
torch.manual_seed(random_seed)
```

```
# 代码3
# 加载数据集
# 数据预处理和加载
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,)) # MNIST 标准化参数
])

# 加载 MNIST 数据集, 使用 transforms 进行数据预处理和标准化
train_dataset = datasets.MNIST('./data', train=True, download=True, transform=transform)
test_dataset = datasets.MNIST('./data', train=False, download=True, transform=transform)

# 创建数据加载器
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)
```

基于卷积神经网络的手写数字识别

➤ 2. 数据加载与预处理

再次，以test_loader为例，观察数据集的基本概貌，其中，example_targets是图像实际对应的数字标签。

最后，使用Matplotlib绘制其中的6张数字图像。



```
# 代码4
# 观察数据形状
examples = enumerate(test_loader)
batch_idx, (example_data, example_targets) = next(examples)
print(example_targets)
print(example_data.shape)
```

```
tensor([7, 2, 1, 0, 4, 1, 4, 9, 5, 9, 0, 6, 9, 0, 1, 5, 9, 7, 3, 4, 9, 6, 6, 5,
        4, 0, 7, 4, 0, 1, 3, 1, 3, 4, 7, 2, 7, 1, 2, 1, 1, 7, 4, 2, 3, 5, 1, 2,
        4, 4, 6, 3, 5, 5, 6, 0, 4, 1, 9, 5, 7, 8, 9, 3])
torch.Size([64, 1, 28, 28])
```

```
# 代码5
# 绘制图片
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams['font.family']='SimHei'
fig = plt.figure()
for i in range(6):
    plt.subplot(2, 3, i + 1)
    plt.tight_layout()
    plt.imshow(example_data[i][0], cmap='gray', interpolation='none')
    plt.title('目标数据标签: {}'.format(example_targets[i]))
    plt.xticks([])
    plt.yticks([])
plt.show()
```

基于卷积神经网络的手写数字识别

➤ 3. 构建网络

构建卷积神经网络时，使用两个Conv2d卷积层和两个全连接层Linear，并且选择整流线性单元（简称ReLU）作为激活函数以及使用两个Dropout层。

首先，导入相关库。

然后，为构建的网络创建一个新类Net。forward函数定义了使用给定的层和函数计算输出。

最后，初始化网络和优化器。

```
# 代码6
# 导入相关库
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
```

```
# 代码7
# 创建一个新类Net
class Net(nn.Module):

    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(1, 10, kernel_size=5)
        self.conv2 = nn.Conv2d(10, 20, kernel_size=5)
        self.conv2_drop = nn.Dropout2d()
        self.fc1 = nn.Linear(320, 50)
        self.fc2 = nn.Linear(50, 10)

    def forward(self, x):
        x = F.relu(F.max_pool2d(self.conv1(x), 2))
        x = F.relu(F.max_pool2d(self.conv2_drop(self.conv2(x)), 2))
        x = x.view(-1, 320)
        x = F.relu(self.fc1(x))
        x = F.dropout(x, training=self.training)
        x = self.fc2(x)
        return F.log_softmax(x, dim=1)
```

```
# 代码8
# 初始化网络和优化器
network = Net()
optimizer=optim.SGD(network.parameters(), lr=learning_rate, momentum=momentum)
```

基于卷积神经网络的手写数字识别

➤ 4. 训练网络

在训练网络过程中，分别定义训练函数和测试函数。通过训练函数，观察使用随机初始化的网络参数获得的精度和损失率。在测试函数中跟踪正确分类的数字来计算网络的精度。每一个epoch对所有训练数据进行一次迭代，将在循环遍历n_epochs之前手动添加测试函数调用，以便运用随机初始化的参数来评估模型。

首先，设置参数train_losses、train_counter、test_losses和test_counter并初始化。

其次，进行一次测试循环。首先，自定义训练函数train，调用optimizer.zero_grad()方法手动将梯度设置为零，生成网络的输出并计算输出与真值标签之间的负对数概率损失。并使用optimizer.step()方法收集一组新的梯度并传播回每个网络参数，使用state_dict()方法保存神经网络模块以及优化器的内部状态。通过1次测试，观察使用随机初始化的网络参数获得的精度和损失率。

```
# 代码9
# 参数设置
train_losses = []
train_counter = []
test_losses = []
test_counter = [i * len(train_loader.dataset) for i in range(0, n_epochs + 1)]
```

```
# 代码10
# 一次测试循环
def train(epoch):
    network.train()
    for batch_idx, (data, target) in enumerate(train_loader):
        optimizer.zero_grad()
        output = network(data)
        loss = F.nll_loss(output, target)
        loss.backward()
        optimizer.step()
        if batch_idx % log_interval == 0:
            print('Train Epoch: {} [{}/{} ({:.0f}%)]\tLoss: {:.6f}'.format(
                epoch, batch_idx * len(data), len(train_loader.dataset),
                100. * batch_idx / len(train_loader), loss.item()))
            train_losses.append(loss.item())
            train_counter.append(
                (batch_idx * 64) + ((epoch - 1) * len(train_loader.dataset)))
            torch.save(network.state_dict(), './tmp/model.pth')
            torch.save(optimizer.state_dict(), './tmp/optimizer.pth')

train(1)
```

```
Train Epoch: 1 [0/60000 (0%)] Loss: 2.309155
Train Epoch: 1 [640/60000 (1%)] Loss: 2.308265
Train Epoch: 1 [1280/60000 (2%)] Loss: 2.231261
Train Epoch: 1 [1920/60000 (3%)] Loss: 2.266338
Train Epoch: 1 [2560/60000 (4%)] Loss: 2.222587
Train Epoch: 1 [3200/60000 (5%)] Loss: 2.166098
```

基于卷积神经网络的手写数字识别

➤ 4. 训练网络

然后，进入测试循环，总结总体的测试损失，并跟踪正确分类的数字来计算网络的精度。首先，定义测试方法test()，在训练网络过程中，使用上下文管理器no_grad()方法避免将生成网络输出的计算结果存储在计算图中。然后，调用测试方法test()进行测试。

最后，循环n_epochs次训练进行模型评估，每训练完一个周期调用一次测试方法test()。

```
Test set: Avg. loss: 0.1863, Accuracy: 9445/10000 (94%)
```

```
Train Epoch: 1 [0/60000 (0%)] Loss: 0.309545
Train Epoch: 1 [640/60000 (1%)] Loss: 0.745244
Train Epoch: 1 [1280/60000 (2%)] Loss: 0.448738
Train Epoch: 1 [1920/60000 (3%)] Loss: 0.725766
Train Epoch: 1 [2560/60000 (4%)] Loss: 0.450177
Train Epoch: 1 [3200/60000 (5%)] Loss: 0.468146
Train Epoch: 1 [3840/60000 (6%)] Loss: 0.470779
Train Epoch: 1 [4480/60000 (7%)] Loss: 0.367244
```

```
Train Epoch: 3 [57600/60000 (96%)] Loss: 0.197039
Train Epoch: 3 [58240/60000 (97%)] Loss: 0.165374
Train Epoch: 3 [58880/60000 (98%)] Loss: 0.297370
Train Epoch: 3 [59520/60000 (99%)] Loss: 0.201526
```

```
Test set: Avg. loss: 0.0808, Accuracy: 9733/10000 (97%)
```

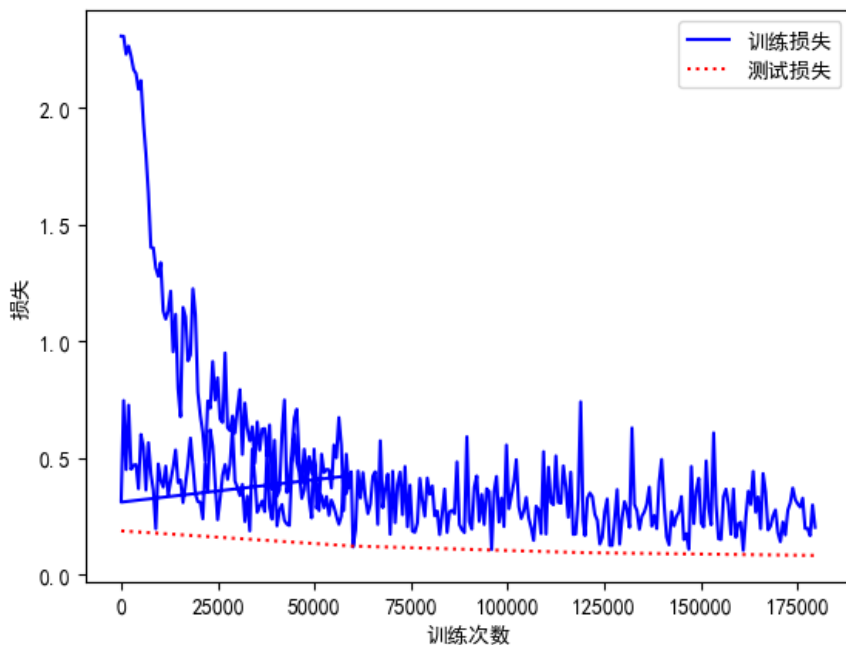
```
# 代码11
# 测试循环
def test():
    network.eval()
    test_loss = 0
    correct = 0
    with torch.no_grad():
        for data, target in test_loader:
            output = network(data)
            test_loss += F.nll_loss(output, target, reduction='sum').item()
            pred = output.data.max(1, keepdim=True)[1]
            correct += pred.eq(target.data.view_as(pred)).sum()
    test_loss /= len(test_loader.dataset)
    test_losses.append(test_loss)
    print('\nTest set: Avg. loss: {:.4f}, Accuracy: {}/{} ({:.0f}%)\n'.format(
        test_loss, correct, len(test_loader.dataset),
        100. * correct / len(test_loader.dataset)))
```

```
# 代码12
# 模型评估
test()
for epoch in range(1, n_epochs + 1):
    train(epoch)
    test()
```

基于卷积神经网络的手写数字识别

➤ 5. 性能评估

通过前面介绍，仅仅经过3个阶段的训练，已经能够达到测试集97%的准确率。下面通过画出训练中损失的变化曲线，观察训练过程的参数变化，对模型性能进行评估。



```
# 代码13
# 性能评估
import matplotlib.pyplot as plt

fig = plt.figure()
plt.plot(train_counter, train_losses, color='blue')
plt.plot(test_counter, test_losses, color='red', linestyle=':')
plt.legend(['训练损失', '测试损失'], loc='upper right')
plt.xlabel('训练次数')
plt.ylabel('损失')
plt.savefig('./tmp/1.png', dpi=1080)
plt.show()
```